

	ITS Computer-Vision auf dem ESP32 P4		
	Nachname, Vorname:	Klasse:	Datum:

esp-nn: Die Kraftmaschine (Low-Level)

esp-nn steht für "Espressif Neural Network" und ist eine Bibliothek, die sich auf die **maximale Optimierung der Basisfunktionen** konzentriert.

- **Was es tut:** Es enthält hochoptimierte Implementierungen für neuronale Netzwerk-Kernel (z. B. Convolution, Pooling, Softmax, Aktivierungsfunktionen).
- **Der Clou:** Diese Funktionen sind oft in **Assembly** oder mit speziellen **Intrinsics** geschrieben, um die Hardware-Beschleuniger der ESP32-Chips (wie die AI-Instruktionen des S3) direkt anzusprechen.
- **Für wen:** Normalerweise nutzt du esp-nn nicht direkt, es sei denn, du schreibst dein eigenes KI-Framework oder möchtest eine extrem spezifische mathematische Operation beschleunigen. Es dient primär als **Backend** für andere Bibliotheken (wie TensorFlow Lite Micro oder eben esp-dl).

esp-dl: Das Framework (High-Level)

esp-dl steht für "Espressif Deep Learning" und ist eine vollständige Bibliothek für die **Entwicklung und Bereitstellung von KI-Modellen**.

- **Was es tut:** Es bietet fertige Layer, Modellstrukturen und Werkzeuge, um ein komplettes Modell (z. B. zur Gesichtserkennung oder Sprachsteuerung) auf dem Chip laufen zu lassen. Es gibt ein spezielle espdl KI-Modellformat.
- **Der Clou:** esp-dl nutzt im Hintergrund esp-nn, um schnell zu sein, kümmert sich aber um das "Drumherum": Speicherverwaltung, Modell-Laden und die Verkettung der Schichten.
- **Features:** Es enthält oft bereits vortrainierte Modelle (wie Human Face Detection oder Wake Word Detection) und Tools zur **Quantisierung** (um Modelle von 32-Bit auf 8-Bit zu schrumpfen). Unter <https://github.com/espressif/esp-dl> gibt es unter anderem momentan folgende vortrainierte Modelle:
 - Pedestrian Detection
 - Human Face Recognition
 - Imagenet Classification (MobileNetV2)
 - COCO Detection (YOLO11n)
 - Cat Detection (ESPDet-Pico)
 - Dog Detection (ESPDet-Pico)
 - Hand Detection (ESPDet-Pico)
 - Hand Gesture Recognition
 - Pose Estimation (YOLO11n-Pose)
 - Speaker Verification (x-vector)

Es ist auch möglich, beliebige andere Modelle in esp-dl zu konvertieren. Espressif liefert dazu Tools, die beliebige Modelle vom ONNX-Format ins esp-dl Format umwandeln. Das ONNX-Format ist ein Austauschformat, in das fast alle Modelle konvertiert werden können.

TensorFlow Lite Micro: Das Standard-Inferenzmodell

- **Was es tut:** TensorFlow Lite (TFLite) ist eine Open-Source-Deep-Learning-Framework-Erweiterung von Google, die speziell für das Ausführen von Machine-Learning-Modellen auf Endgeräten (On-Device Inference / Edge AI) entwickelt wurde. Im Gegensatz zum klassischen TensorFlow, das meist auf leistungsstarken Servern oder in der Cloud läuft, ist TFLite für ressourcenbeschränkte Umgebungen wie Smartphones, IoT-Geräte, eingebettete Systeme und Mikrocontroller optimiert.
- **Der Clou:** Eine spezielle Untermenge von TFLite ist TFLite Micro. Diese Variante ist so konzipiert, dass sie selbst auf digitalen Signalprozessoren (DSPs) und winzigen Mikrocontrollern (z. B. ARM Cortex-M, ESP32) läuft. Sie benötigt kein Betriebssystem (Bare-Metal), kommt mit wenigen Kilobytes Speicher aus und ermöglicht "TinyML"-Anwendungen direkt auf der Hardware.
- **Features:** Jedes Tensorflow Modell kann im Prinzip (!Größe) auf ESP32 implementiert werden. Tensorflow ist allerdings auf dem ESP32 bei weitem nicht so optimiert wie esp-dl.

	ITS Computer-Vision auf dem ESP32 P4		
	Nachname, Vorname:	Klasse:	Datum:

Ein wichtiger Schritt bei der Konvertierung für den ESP32 ist die Quantisierung. Die Gründe dafür sind:

Speicherplatz: Ein originaler Modell-Tensor in 32-Bit-Fließkomma (FP32) belegt viel Platz. Durch die Reduzierung auf 8-Bit-Ganzzahlen (INT8) schrumpft das Modell im Flash-Speicher um **75 %**.

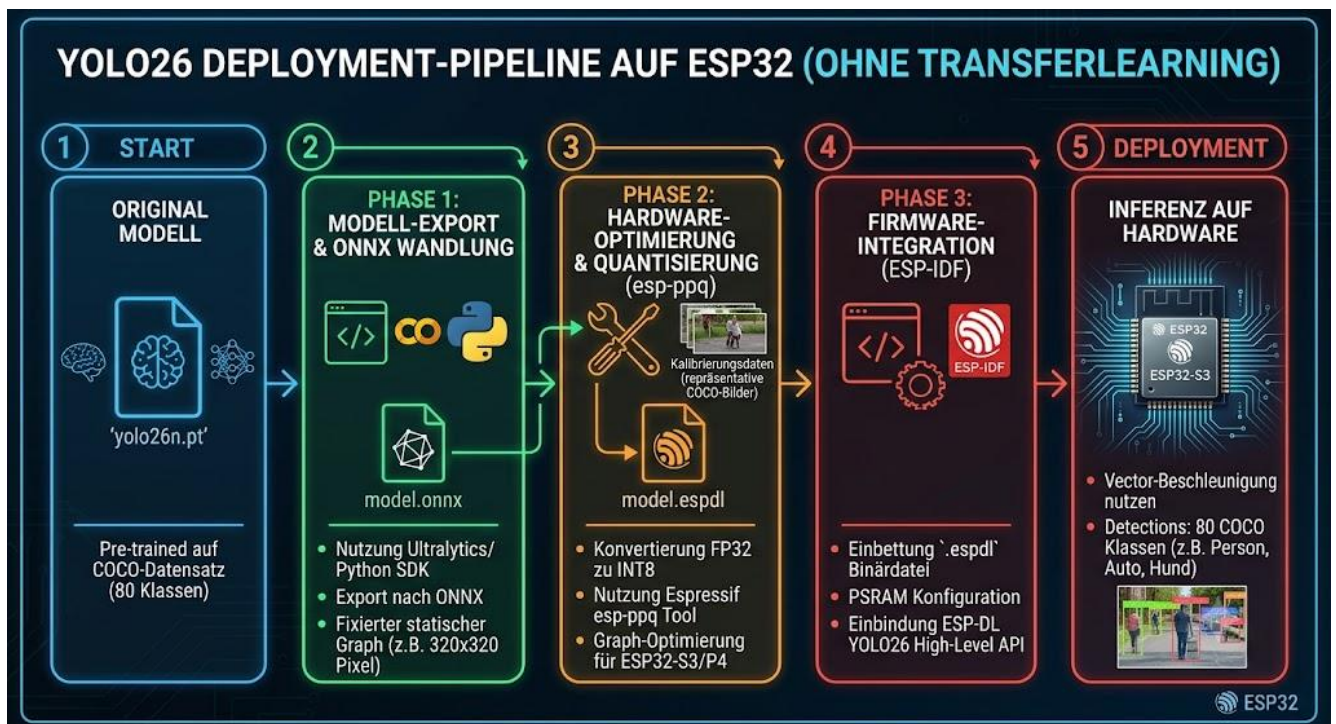
PDF+ 3

Performance: Die Hardware-Beschleuniger (Vektorbefehle) des ESP32-S3 und P4 können INT8-Ganzzahlen um ein Vielfaches schneller verarbeiten als komplexe Fließkommazahlen.

Dabei spielt der **PSRAM** eine wichtige Rolle:

Während der Inferenz müssen riesige Zwischenergebnisse (Tensoren) im Arbeitsspeicher gehalten werden. Der interne SRAM des ESP32 ist dafür meist zu klein. Daher muss im sdkconfig-Menü des **ESP-IDF** der externe PSRAM (Pseudo-SRAM) explizit aktiviert werden, um Speicherüberläufe (*Out-of-Memory*) zu verhindern.

Die folgende Abbildung veranschaulicht den Weg von einem Standard-Modell im offenen ONNX-Format zu einer Lauffähigen Anwendung auf dem ESP32.



Fragen

a) Erkläre den Unterschied in der Aufgabenstellung zwischen den beiden Bibliotheken esp-nn und esp-dl. Wie arbeiten sie zusammen?

b) Ein Entwickler hat ein KI-Modell in einem gängigen Drittanbieter-Framework trainiert. Beschreibe das standardisierte Verfahren, um dieses Modell auf dem ESP32 lauffähig zu machen.

	ITS			Computer-Vision auf dem ESP32 P4		
	Nachname, Vorname:			Klasse:		Datum:

c) In Phase 2 der Deployment-Pipeline werden "repräsentative COCO-Bilder" als Kalibrierungsdaten eingesetzt. Welche technische Transformation wird in dieser Phase durchgeführt und warum sind die Bilder dafür notwendig?

d) Unter welchen spezifischen Bedingungen ist es sinnvoll, direkt mit der Bibliothek esp-nn zu arbeiten?

e) Welche konkreten Daten liefert das fertig integrierte YOLO26n-Modell nach der Inferenz auf der Hardware an das System zurück?

f) Warum sollte man auf dem ESP32 ESP-DL dem Tensorflow-Framework vorziehen?

g) Wozu dient das ONNX-Format?